VOL. 36

Python for AI

Python programming for data science and AI

Source	kaggle.com/learn/python
Generated	2026-10-11
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 36 - Python for AI

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter 1 - Python basics 8

1.1 Summary 8

1.2 Concepts 8

Variables and naming 9

The print function 9

1.3 Examples 9

Example 1 - Variable assignment 9

Example 2 - Arithmetic 9

Example 3 - String formatting 10

1.4 Common mistakes and best practices 10

1.5 Practice exercises 11

1.6 Solutions 11

1.7 Mini test 11

Questions 11

Answer key and explanations 11

Chapter 2 - Conditionals and loops 11

2.1 Summary 11

2.2 Concepts 12

Comparison operators 12

Logical operators 12

2.3 Examples 12

Example 1 - If / elif / else 12

Example 2 - For loop 13

Example 3 - While loop with break 13

2.4 Common mistakes and best practices 13

2.5 Practice exercises 14

2.6 Solutions 14

2.7 Mini test 14

Questions 14

Answer key and explanations 14

Chapter 3 - Functions 15

3.1 Summary . . . 15
3.2 Concepts . . . 15
Defining and calling functions . . . 15
Default arguments . . . 15
Lambda functions . . . 15
3.3 Examples . . . 15
Example 1 - Multiple return values . . . 16
Example 2 - Using helper functions . . . 16
Example 3 - Lambda with sorted . . . 16
3.4 Common mistakes and best practices . . . 17
3.5 Practice exercises . . . 17
3.6 Solutions . . . 17
3.7 Mini test . . . 17
Questions . . . 17
Answer key and explanations . . . 18
Chapter 4 - Lists and tuples . . . 18
4.1 Summary . . . 18
4.2 Concepts . . . 18
List indexing . . . 18
Slicing . . . 18
4.3 Examples . . . 19
Example 1 - List methods . . . 19
Example 2 - List comprehensions . . . 19
Example 3 - Tuples and swapping . . . 20
4.4 Common mistakes and best practices . . . 20
4.5 Practice exercises . . . 20
4.6 Solutions . . . 21
4.7 Mini test . . . 21
Questions . . . 21
Answer key and explanations . . . 21
Chapter 5 - Dictionaries . . . 21
5.1 Summary . . . 21
5.2 Concepts . . . 22
Creating dictionaries . . . 22
5.3 Examples . . . 22

Example 1 - Basic operations . . . 22
Example 2 - Iterating over dictionaries . . . 22
Example 3 - Nested dictionaries . . . 23
5.4 Common mistakes and best practices . . . 24
5.5 Practice exercises . . . 24
5.6 Solutions . . . 24
5.7 Mini test . . . 24
Questions . . . 24
Answer key and explanations . . . 24
Chapter 6 - Sets and data cleaning . . . 25
6.1 Summary . . . 25
6.2 Concepts . . . 25
Creating sets . . . 25
6.3 Examples . . . 25
Example 1 - Set operations . . . 25
Example 2 - Removing duplicates from data . . . 26
Example 3 - Cleaning with pandas . . . 26
6.4 Common mistakes and best practices . . . 27
6.5 Practice exercises . . . 27
6.6 Solutions . . . 28
6.7 Mini test . . . 28
Questions . . . 28
Answer key and explanations . . . 28
Chapter 7 - String manipulation . . . 28
7.1 Summary . . . 28
7.2 Concepts . . . 29
String methods . . . 29
F-strings . . . 29
7.3 Examples . . . 29
Example 1 - Parsing structured strings . . . 29
Example 2 - F-strings with formatting . . . 29
Example 3 - Regular expressions . . . 30
7.4 Common mistakes and best practices . . . 30
7.5 Practice exercises . . . 31
7.6 Solutions . . . 31

7.7 Mini test . . . 31
Questions . . . 31
Answer key and explanations . . . 31
Chapter 8 - Advanced Python . . . 31
8.1 Summary . . . 31
8.2 Concepts . . . 32
Exception handling . . . 32
8.3 Examples . . . 32
Example 1 - Context managers for file I/O . . . 32
Example 2 - Working with modules . . . 32
Example 3 - Custom exception . . . 33
8.4 Common mistakes and best practices . . . 33
8.5 Practice exercises . . . 34
8.6 Solutions . . . 34
8.7 Mini test . . . 34
Questions . . . 34
Answer key and explanations . . . 34
Chapter 9 - Data science with pandas . . . 35
9.1 Summary . . . 35
9.2 Concepts . . . 35
What is a DataFrame? . . . 35
9.3 Examples . . . 35
Example 1 - Loading and inspecting data . . . 35
Example 2 - Filtering and grouping . . . 36
Example 3 - Handling missing data . . . 37
9.4 Common mistakes and best practices . . . 38
9.5 Practice exercises . . . 39
9.6 Solutions . . . 39
9.7 Mini test . . . 39
Questions . . . 39
Answer key and explanations . . . 39
Chapter 10 - Numpy arrays . . . 39
10.1 Summary . . . 39
10.2 Concepts . . . 40
Creating arrays . . . 40

10.3 Examples . . . 40
Example 1 - Vectorised operations . . . 40
Example 2 - 2D arrays and slicing . . . 41
Example 3 - Broadcasting . . . 41
10.4 Common mistakes and best practices . . . 42
10.5 Practice exercises . . . 43
10.6 Solutions . . . 43
10.7 Mini test . . . 43
Questions . . . 43
Answer key and explanations . . . 43
Chapter 11 - Data visualisation . . . 43
11.1 Summary . . . 44
11.2 Concepts . . . 44
The matplotlib object model . . . 44
11.3 Examples . . . 44
Example 1 - Line plot . . . 44
Example 2 - Histogram . . . 45
Example 3 - Scatter plot with seaborn . . . 45
Example 4 - Bar chart . . . 45
11.4 Common mistakes and best practices . . . 46
11.5 Practice exercises . . . 46
11.6 Solutions . . . 46
11.7 Mini test . . . 47
Questions . . . 47
Answer key and explanations . . . 47
Chapter 12 - Feature engineering . . . 47
12.1 Summary . . . 47
12.2 Concepts . . . 47
Scaling numerical features . . . 47
12.3 Examples . . . 47
Example 1 - One-hot encoding . . . 48
Example 2 - Scaling features . . . 48
Example 3 - Missing value imputation . . . 49
12.4 Common mistakes and best practices . . . 50
12.5 Practice exercises . . . 51

12.6 Solutions . . . 51
12.7 Mini test . . . 51
Questions . . . 51
Answer key and explanations . . . 51
Chapter 13 - End-to-end project . . . 51
13.1 Summary . . . 51
13.2 Concepts . . . 52
The ML workflow . . . 52
13.3 Examples . . . 52
Example 1 - Loading and exploring a dataset . . . 52
Example 2 - Feature engineering and model training . . . 53
Example 3 - Submitting predictions . . . 54
13.4 Common mistakes and best practices . . . 55
13.5 Practice exercises . . . 55
13.6 Solutions . . . 56
13.7 Mini test . . . 56
Questions . . . 56
Answer key and explanations . . . 56
Chapter 14 - Where to go next . . . 56
14.1 Summary . . . 56
14.2 Concepts . . . 57
Continuing the journey . . . 57
14.3 Examples . . . 57
Example 1 - Decorators (advanced) . . . 57
14.4 Common mistakes . . . 57
14.5 Practice exercises . . . 58
14.6 Mini test . . . 58
Questions . . . 58
Answer key and explanations . . . 58
Cheat sheet - Python for data science . . . 58
15.1 Libraries quick reference . . . 59
Final test . . . 59
16.1 Final test answers . . . 60
Sources . . . 60

How to use this volume

Prerequisites. No prior programming experience required. This volume starts from the very beginning.

Estimated study time. About 20 hours: one to two hours per chapter, plus the time you spend typing the examples yourself.

How the chapters work. Every chapter opens with a summary, explains concepts with tables and diagrams, shows numbered examples with real output, lists common mistakes with best practices, and closes with practice exercises, solutions, and a mini test with an answer key.

How to run the code. Every Python example was executed with Python 3.13 on this machine. Save an example in a file and run it with `python filename.py`. Blocks tagged `python` are complete programs, `text` shows output, and `bash` shows terminal commands.

shell

```
python --version
python ch03_variables.py
```

output

```
Python 3.13.15
x = 10
y = 3.14
```

About the source. The syllabus follows the free Kaggle Learn "Python" course. All explanations, examples, exercises and tests are original writing; the course material itself was never copied.

Chapter 1 - Python basics

1.1 Summary

- Python is a high-level, interpreted language popular for data science and AI.
- Variables store values that can be reused throughout a program.
- Types include integers, floats, strings and booleans.
- Arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) work as expected.
- `print()` displays output to the screen.
- Python uses indentation (spaces, not tabs) to group code.
- Comments start with `#`.
- Input from the user comes via `input()`.

1.2 Concepts

Python programs are read top to bottom, left to right. Every line is either a statement or an expression.

Variables and naming

A variable name can contain letters, numbers and underscores, but cannot start with a number.

Invalid	Valid
`2var`	`var2`
`my-var`	`my_var`
`for` (reserved word)	`force`

The print function

The `print()` function outputs text or values to the console:

Python

```python
# ch01_print.py
print("Hello, World!")
print(42)
print("The answer is", 42)
```

output

```
Hello, World!
42
The answer is 42
```

1.3 Examples

Example 1 - Variable assignment

Python

```python
# ch01_variables.py
name = "Ada"
age = 36
is_programmer = True

print(f"Name: {name}")
print(f"Age: {age}")
print(f"Is programmer: {is_programmer}")
```

output

```
Name: Ada
Age: 36
Is programmer: True
```

Example 2 - Arithmetic

Python

```python
# ch01_arithmetic.py
a = 10
b = 3

print(f"a + b = {a + b}")
print(f"a - b = {a - b}")
print(f"a * b = {a * b}")
print(f"a / b  = {a / b}")  # always a float
print(f"a // b = {a // b}")  # integer division
print(f"a % b  = {a % b}")   # remainder
print(f"a ** b = {a ** b}")  # exponentiation
```

output

```
a + b = 13
a - b = 7
a * b = 30
a / b  = 3.3333333333333335
a // b = 3
a % b  = 1
a ** b = 1000
```

Example 3 - String formatting

Python

```python
# ch01_strings.py
greeting = "Hello"
name = "Grace"
full = f"{greeting}, {name}!"
print(full)
print(f"Length: {len(full)}")
print(full.upper())
print(full.lower())
```

output

```
Hello, Grace!
Length: 13
HELLO, GRACE!
hello, grace!
```

1.4 Common mistakes and best practices

Mistake	What happens	Best practice
Using = instead of == for comparison	SyntaxError or unexpected behaviour	Use == for equality tests
Not quoting strings	`hello` treated as a variable name, causes NameError	Always use `'hello'` or `"hello"`
Mixing tabs and spaces	IndentationError	Use 4 spaces consistently
Forgetting `print()`	Nothing appears in the output	Use `print()` to see values

1.5 Practice exercises

1. Assign your name to a variable and print it.
2. Write a program that converts Celsius to Fahrenheit. Formula: `F = C * 9/5 + 32`.
3. Calculate the area of a circle: `area = pi * r**2`. Use `pi = 3.14159`.
4. Swap two variables without using a temporary variable: `a, b = b, a`.
5. Type `type()` on any value—what does it return?

1.6 Solutions

1. `name = "Alice"; print(name)`
2. `c = 25; f = c * 9/5 + 32; print(f"We are {f} degrees Fahrenheit.")`
3. `r = 5; pi = 3.14159; area = pi * r**2; print(f"Area: {area}")`
4. `a, b = 1, 2; a, b = b, a; print(f"a={a}, b={b}")`
5. `type()` returns the data type of the value.

1.7 Mini test

Questions

1. What is the output of `print(10 / 2)`?
2. What does `input()` return?
3. Which operator gives the remainder of a division?
4. What happens if you use `Print()` instead of `print()`?
5. How many spaces should you use for indentation in Python?

Answer key and explanations

1. `5.0` — Division always returns a float, even when the result is a whole number.
2. A string — `input()` reads a line from standard input and returns it as text.
3. `%` — The modulo operator returns the remainder.
4. `NameError: name 'Print' is not defined` — Python is case-sensitive.
5. Four spaces — This is the PEP 8 standard.

Chapter 2 - Conditionals and loops

2.1 Summary

- `if / elif / else` makes decisions in your code.
- Comparisons (==, `!=`, <, >, <=, >=) return booleans.
- `and`, `or`, `not` combine boolean expressions.

- `for` loops iterate over a sequence or range.
- `while` loops repeat while a condition is true.
- `break` stops a loop; `continue` skips to the next iteration.
- Indentation determines what code belongs to which branch or loop.

2.2 Concepts

Comparison operators

Operator	Meaning
`==`	equal to
`!=`	not equal to
`<`	less than
`>`	greater than
`<=`	less than or equal
`>=`	greater than or equal

Logical operators

Operator	Meaning
`and`	both conditions must be true
`or`	at least one condition must be true
`not`	negates a boolean

2.3 Examples

Example 1 - If / elif / else

Python

```python
# ch02_grades.py
score = 85

if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
else:
    grade = 'F'

print(f"Score: {score}, Grade: {grade}")
```

output

```
Score: 85, Grade: B
```

Example 2 - For loop

Python

```python
# ch02_for_loop.py
total = 0
for i in range(1, 11):
    total += i
    print(f"i={i}, total={total}")
print(f"Sum of 1 to 10 is {total}")
```

output

```
i=1, total=1
i=2, total=3
i=3, total=6
i=4, total=10
i=5, total=15
i=6, total=21
i=7, total=28
i=8, total=36
i=9, total=45
i=10, total=55
Sum of 1 to 10 is 55
```

Example 3 - While loop with break

Python

```python
# ch02_while_break.py
count = 0
while True:
    count += 1
    print(f"Iteration {count}")
    if count >= 3:
        print("Reached limit, stopping.")
        break
```

output

```
Iteration 1
Iteration 2
Iteration 3
Reached limit, stopping.
```

2.4 Common mistakes and best practices

Mistake	What happens	Best practice
Forgetting : at the end of if/for/while	SyntaxError	Always add :
Not indenting the body	IndentationError	Indent 4 spaces

Mistake	What happens	Best practice
Infinite loop	Program hangs	Make sure the condition can become false
Using `is` instead of `==` for value comparison	Wrong result	Use `==` for values, `is` for `None`/`True`

2.5 Practice exercises

1. Write a program that checks if a number is positive, negative, or zero.
2. Print the multiplication table from 1 to 10 using a nested `for` loop.
3. Use a `while` loop to count down from 5 to 1.
4. Write a program that sums all even numbers from 1 to 100.
5. Use `continue` to skip odd numbers when printing 1 to 10.

2.6 Solutions

1. `if n > 0: print("positive") elif n < 0: print("negative") else: print("zero")`
2. Nested loops with `i * j` for each combination.
3. `while n > 0: print(n); n -= 1`
4. `for n in range(1,101): if n % 2 == 0: s += n`
5. `for n in range(1,11): if n % 2 != 0: continue; print(n)`

2.7 Mini test

Questions

1. What does `range(5)` produce?
2. Which operator checks if two values are NOT equal?
3. What is the output of `if True or False: print("yes") else: print("no")`?
4. What does `break` do inside a loop?
5. When should you use `while` vs `for`?

Answer key and explanations

1. `0, 1, 2, 3, 4` — `range(n)` produces numbers from 0 to n-1.
2. `!=` — The not-equal operator.
3. `yes` — `or` returns True if either side is True.
4. `break` stops the loop immediately and exits to the next statement.
5. Use `for` when you know the number of iterations; use `while` when the loop depends on a condition that changes.

Chapter 3 - Functions

3.1 Summary

- A function is a named block of code that performs a specific task.
- Define with `def`, call by name with parentheses.
- Functions can take arguments (parameters) and return values.
- `return` exits the function and passes a value back.
- Arguments can have default values.
- Lambda functions are small, anonymous functions on one line.
- Scope: local variables are only visible inside the function.

3.2 Concepts

Defining and calling functions

Python

```python
def greet(name):
    message = f"Hello, {name}!"
    return message

result = greet("Alan")
print(result)
```

Default arguments

Python

```python
def power(base, exponent=2):
    return base ** exponent

print(power(3))      # 9
print(power(2, 10))  # 1024
```

Lambda functions

Python

```python
square = lambda x: x * x
print(square(5))  # 25

# Often used with map, filter, sort
numbers = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, numbers))
print(doubled)  # [2, 4, 6, 8]
```

3.3 Examples

Example 1 - Multiple return values

Python

```python
# ch03_returns.py
def divide_and_remainder(a, b):
    quotient = a // b
    remainder = a % b
    return quotient, remainder

q, r = divide_and_reainder(17, 5)
print(f"17 / 5: quotient={q}, remainder={r}")
```

output

```
17 / 5: quotient=3, remainder=2
```

Example 2 - Using helper functions

Python

```python
# ch03_helpers.py
def celsius_to_fahrenheit(c):
    return c * 9/5 + 32

def categorize(temp_f):
    if temp_f < 32:
        return "freezing"
    elif temp_f < 50:
        return "cold"
    elif temp_f < 70:
        return "cool"
    else:
        return "warm or hot"

temps_c = [0, 10, 20, 30, 40]
for c in temps_c:
    f = celsius_to_fahrenheit(c)
    category = categorize(f)
    print(f"{c}C -> {f}F ({category})")
```

output

```
0C -> 32.0F (freezing)
10C -> 50.0F (cold)
20C -> 68.0F (cool)
30C -> 86.0F (warm or hot)
40C -> 104.0F (warm or hot)
```

Example 3 - Lambda with sorted

Python

```python
# ch03_lambda_sort.py
students = [("Alice", 85), ("Bob", 92), ("Carol", 78)]

# Sort by score (highest first)
by_score = sorted(students, key=lambda s: s[1], reverse=True)
print("By score:", by_score)

# Sort by name
by_name = sorted(students, key=lambda s: s[0])
print("By name:", by_name)
```

output

```
By score: [('Bob', 92), ('Alice', 85), ('Carol', 78)]
By name: [('Alice', 85), ('Bob', 92), ('Carol', 78)]
```

3.4 Common mistakes and best practices

Mistake	What happens	Best practice
Forgetting `return`	Function returns `None`	Always return what the function should output
Modifying a global variable	Confusing side effects	Use parameters and return values
Too many arguments	Hard to read and call	Limit to 3-4; use dicts or classes for more
Side effects in functions	Unexpected behaviour	Make functions pure when possible

3.5 Practice exercises

1. Write a function that returns the factorial of n (n!).
2. Write a function that checks if a string is a palindrome.
3. Write a function that takes a list and returns only the even numbers.
4. Write a lambda that takes two numbers and returns the larger.
5. Write a function with a default argument: `def greet(name, greeting="Hello")`.

3.6 Solutions

1. Use recursion or a loop: `if n <= 1: return 1; return n * factorial(n-1)`
2. `return s == s[::-1]`
3. `return [x for x in lst if x % 2 == 0]`
4. `max_of_two = lambda a, b: a if a > b else b`
5. `return f"{greeting}, {name}!"`

3.7 Mini test

Questions

1. What keyword defines a function in Python?
2. What does `return` do?
3. What is the default return value of a function without `return`?
4. What is a lambda function?
5. What is the lifetime of a local variable?

Answer key and explanations

1. `def` — Example: `def my_function():`
2. `return` exits the function and passes a value back to the caller.
3. `None` — Python's null value.
4. A small, anonymous function written on one line using `lambda`.
5. It exists only while the function is running; it is created on entry and destroyed on exit.

Chapter 4 - Lists and tuples

4.1 Summary

- Lists are ordered, mutable collections: `[1, 2, 3]`.
- Tuples are ordered, immutable collections: `(1, 2, 3)`.
- Index with `[0]`, negative indices from the end.
- Slice with `[start:stop:step]`.
- Common methods: `append`, `extend`, `insert`, `remove`, `pop`, `sort`, `copy`.
- List comprehensions create new lists concisely: `[x*2 for x in range(5)]`.
- Lists can hold mixed types; tuples are often used for heterogeneous data.

4.2 Concepts

List indexing

Python

```python
shopping = ["apples", "bread", "milk"]
print(shopping[0])    # apples
print(shopping[-1])   # milk (last item)
```

Slicing

Python

```python
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4])      # [1, 2, 3]
print(nums[:3])       # [0, 1, 2]
print(nums[3:])       # [3, 4, 5]
print(nums[::2])      # [0, 2, 4] (every second)
print(nums[::-1])     # [5, 4, 3, 2, 1, 0] (reversed)
```

4.3 Examples

Example 1 - List methods

Python

```python
# ch04_list_methods.py
tasks = ["write report", "send email"]

# Add items
tasks.append("schedule meeting")
tasks.insert(1, "review draft")
print("After adding:", tasks)

# Remove items
done = tasks.pop(1)  # remove and return item at index 1
print(f"Completed: {done}")
print("Remaining:", tasks)

# Sort and copy
tasks.sort()
backup = tasks.copy()
print("Sorted:", tasks)
print("Backup:", backup)
```

output

```
After adding: ['write report', 'review draft', 'send email', 'schedule meeting']
Completed: review draft
Remaining: ['write report', 'send email', 'schedule meeting']
Sorted: ['schedule meeting', 'send email', 'write report']
Backup: ['schedule meeting', 'send email', 'write report']
```

Example 2 - List comprehensions

Python

```python
# ch04_comprehensions.py
# Squares of numbers 0 to 9
squares = [x**2 for x in range(10)]
print("Squares:", squares)

# Filter even numbers
evens = [x for x in range(20) if x % 2 == 0]
print("Evens:", evens)

# Nested comprehension (5x5 multiplication table)
table = [[i*j for j in range(1, 6)] for i in range(1, 6)]
print("Table row 3:", table[2])
```

output

```
Squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
Evens: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
Table row 3: [3, 6, 9, 12, 15]
```

Example 3 - Tuples and swapping

Python

```python
# ch04_tuples.py
# Tuples are immutable
point = (3, 4)
print("Point:", point)
print("X coordinate:", point[0])

# Tuple unpacking (swap variables)
a = 5
b = 10
a, b = b, a
print(f"After swap: a={a}, b={b}")

# Function returning multiple values (as a tuple)
def stats(data):
    return min(data), max(data), sum(data) / len(data)

scores = [85, 92, 78, 90, 88]
minimum, maximum, average = stats(scores)
print(f"Min: {minimum}, Max: {maximum}, Avg: {average:.1f}")
```

output

```
Point: (3, 4)
X coordinate: 3
After swap: a=10, b=5
Min: 78, Max: 92, Avg: 86.6
```

4.4 Common mistakes and best practices

Mistake	What happens	Best practice
Mutating a list while iterating	Skips elements	Iterate over a copy or build a new list
Forgetting to copy	Two names point to the same list	Use `.copy()` or `list()`
Using a list where a tuple is better	Unnecessary flexibility	Use tuples for fixed data
Index out of range	`IndexError`	Check the length or use `try/except`

4.5 Practice exercises

1. Create a list of 10 random numbers (use `random.randint`).
2. Write a list comprehension that converts a list of Celsius temps to Fahrenheit.

3. Create a tuple of your three favorite colors and unpack them.
4. Given a list with duplicate values, create a new list with only unique values (in order).
5. Use `zip` to combine two lists into a list of pairs.

4.6 Solutions

1. `import random; nums = [random.randint(1,100) for _ in range(10)]`
2. `fahrenheit = [c * 9/5 + 32 for c in celsius]`
3. `colors = ("red", "green", "blue"); c1, c2, c3 = colors`
4. `seen = set(); unique = []; [unique.append(x) for x in lst if x not in seen and not seen.add(x)]`
5. `pairs = list(zip(list1, list2))`

4.7 Mini test

Questions

1. What is the difference between a list and a tuple?
2. What does `lst[1:4]` return for `lst = [0, 1, 2, 3, 4, 5]`?
3. How do you add an item to the end of a list?
4. What does `lst[::-1]` do?
5. What is a list comprehension?

Answer key and explanations

1. Lists are mutable; tuples are immutable.
2. `[1, 2, 3]` — items at indices 1, 2, and 3 (stop is exclusive).
3. `lst.append(item)`.
4. It reverses the list (or any slice step can be negative).
5. A concise way to create a new list by applying an expression to each item in a sequence.

Chapter 5 - Dictionaries

5.1 Summary

- Dictionaries map keys to values: `{"name": "Ada", "age": 36}`.
- Keys must be unique and immutable; values can be any type.
- Fast lookup by key: `d["name"]`.
- Common methods: `keys`, `values`, `items`, `get`, `pop`, `update`.
- `dict.get(key, default)` avoids `KeyError`.

- Iterate with `for key, value in d.items()`.
- Nested dictionaries allow hierarchical data.

5.2 Concepts

Creating dictionaries

Python

```python
# Create an empty dictionary
person = {}

# Create with initial values
person = {
    "name": "Ada",
    "age": 36,
    "is_programmer": True
}

# Using the dict constructor
person = dict(name="Ada", age=36, is_programmer=True)
```

5.3 Examples

Example 1 - Basic operations

Python

```python
# ch05_dict_ops.py
student = {"name": "Alan", "score": 85, "subject": "Math"}

# Add or update a key
student["score"] = 90
student["grade"] = "A"
print("Updated:", student)

# Safe lookup with get
age = student.get("age", 0)  # default value if key doesn't exist
print(f"Age (or default): {age}")

# Remove a key
removed = student.pop("subject")
print(f"Removed: {removed}")
print("After removal:", student)
```

output

```
Updated: {'name': 'Alan', 'score': 90, 'grade': 'A'}
Age (or default): 0
Removed: Math
After removal: {'name': 'Alan', 'score': 90, 'grade': 'A'}
```

Example 2 - Iterating over dictionaries

Python

```python
# ch05_iterate.py
inventory = {
    "apples": 50,
    "bread": 20,
    "milk": 30,
    "eggs": 60
}

# Iterate over keys and values
for item, count in inventory.items():
    print(f"{item}: {count}")

# Sort by key
for item in sorted(inventory):
    print(f"{item}: {inventory[item]}")
```

output

```
apples: 50
bread: 20
milk: 30
eggs: 60
apples: 50
bread: 20
eggs: 60
milk: 30
```

Example 3 - Nested dictionaries

Python

```python
# ch05_nested.py
company = {
    "engineering": {
        "Alice": {"role": "SWE", "years": 3},
        "Bob": {"role": "ML", "years": 2}
    },
    "marketing": {
        "Carol": {"role": "Content", "years": 5}
    }
}

# Access nested values
alice_role = company["engineering"]["Alice"]["role"]
print(f"Alice's role: {alice_role}")

# Safe deep access
def deep_get(d, *keys, default=None):
    for key in keys:
        d = d.get(key, default if isinstance(d, dict) else default)
        if d is None:
            return default
    return d

print("Bob's years:", deep_get(company, "engineering", "Bob", "years", default=0))
print("Unknown:", deep_get(company, "sales", "Nobody", "years", default="N/A"))
```

output

```
Alice's role: SWE
Bob's years: 2
Unknown: N/A
```

5.4 Common mistakes and best practices

Mistake	What happens	Best practice
Using unhashable keys (lists)	`TypeError: unhashable type`	Use tuples or strings as keys
`KeyError` on missing keys	Program crashes	Use `.get()` or `in` checks
Mutating dict during iteration	`RuntimeError`	Iterate over `list(d.items())`
Deeply nested dicts	Hard to maintain	Consider dataclasses or objects

5.5 Practice exercises

1. Create a dictionary mapping country names to their capitals.
2. Count how many times each word appears in a list (word frequency).
3. Merge two dictionaries into one.
4. Create a nested dictionary representing a contact book (name -> phone -> email).
5. Use `dict.get()` to safely look up a key that might not exist.

5.6 Solutions

1. `capitals = {"France": "Paris", "Japan": "Tokyo", "Brazil": "Brasilia"}`
2. `freq = {}; for w in words: freq[w] = freq.get(w, 0) + 1`
3. `merged = {**dict1, **dict2}`
4. Nested dict structure with `in` checks for safety
5. `value = my_dict.get("key", "default_value")`

5.7 Mini test

Questions

1. What makes a valid dictionary key?
2. What is the difference between `d["key"]` and `d.get("key")`?
3. How do you check if a key exists?
4. How do you add a new key-value pair?
5. What method returns all keys in a dictionary?

Answer key and explanations

1. It must be immutable (e.g., string, number, tuple) and unique.

2. `d["key"]` raises `KeyError` if the key doesn't exist; `d.get("key")` returns `None` (or a default).
3. Use `if key in d:` or `if key in d.keys():`.
4. `d["new_key"] = "value"`.
5. `d.keys()` (or just iterate over `d` directly).

Chapter 6 - Sets and data cleaning

6.1 Summary

- Sets are unordered collections of unique items: `{1, 2, 3}`.
- Useful for removing duplicates, membership testing, and set operations.
- Set operations: union, intersection, difference, symmetric difference.
- Sets are mutable; frozensets are immutable.
- Data cleaning involves handling missing values, duplicates and malformed entries.
- Boolean masks and comprehensions make filtering concise.
- Pandas makes data cleaning easy with `dropna`, `fillna`, `drop_duplicates`.

6.2 Concepts

Creating sets

Python

```python
# From a list (duplicates are removed)
numbers = {1, 2, 3, 2, 1, 3, 3}
print(numbers)  # {1, 2, 3}

# Empty set
empty = set()  # NOT {} (that creates an empty dict)

# From any iterable
chars = set("hello")
print(chars)  # {'h', 'e', 'l', 'o'}
```

6.3 Examples

Example 1 - Set operations

Python

```python
# ch06_sets.py
python_users = {"Alice", "Bob", "Charlie", "Diana"}
java_users = {"Bob", "Eve", "Frank", "Diana"}

# Union (both languages)
both_languages = python_users | java_users
print("All users:", both_languages)

# Intersection (both languages)
both = python_users & java_users
print("Both Python and Java:", both)

# Difference (Python only, not Java)
python_only = python_users - java_users
print("Python only:", python_only)

# Symmetric difference (exactly one language)
exactly_one = python_users ^ java_users
print("Exactly one language:", exactly_one)
```

output

```
All users: {'Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'}
Both Python and Java: {'Bob', 'Diana'}
Python only: {'Alice', 'Charlie'}
Exactly one language: {'Alice', 'Charlie', 'Eve', 'Frank'}
```

Example 2 - Removing duplicates from data

Python

```python
# ch06_dedup.py
emails_raw = [
    "alice@example.com",
    "bob@example.com",
    "ALICE@EXAMPLE.COM",
    "charlie@example.com",
    "bob@example.com"
]

# Lowercase + deduplicate
unique_emails = list(dict.fromkeys(
    e.lower() for e in emails_raw
))
print("Unique emails:", unique_emails)

# Membership testing (fast with sets)
forbidden = {"spam@example.com", "bot@example.com"}
valid = [e for e in unique_emails if e not in forbidden]
print("Valid:", valid)
```

output

```
Unique emails: ['alice@example.com', 'bob@example.com', 'charlie@example.com']
Valid: ['alice@example.com', 'bob@example.com', 'charlie@example.com']
```

Example 3 - Cleaning with pandas

Python

```python
# ch06_pandas_cleaning.py
import pandas as pd

# Create a dirty dataset
data = {
    "name": ["Alice", "Bob", None, "Diana", "Eve", "Bob"],
    "age": [25, 30, 35, 28, None, 30],
    "score": [85, 90, 78, None, 92, 90],
    "email": [
        "alice@test.com", "bob@test.com", "bob@test.com",
        "diana@test.com", "eve@test.com", "bob@test.com"
    ]
}
df = pd.DataFrame(data)
print("Raw shape:", df.shape)

# Drop duplicate rows
df = df.drop_duplicates()
print("After dedup:", df.shape)

# Fill missing ages with the mean
df["age"] = df["age"].fillna(df["age"].mean().round())
print("Filled ages:", df["age"].tolist())

# Drop rows where email is missing
df = df.dropna(subset=["email"])
print("Final:", len(df), "rows")
```

output

```
{'name': ['Alice', 'Bob', None, 'Diana', 'Eve', 'Bob'], 'age': [25, 30, 35, 28]... }
Raw shape: (6, 4)
After dedup: (4, 4)
Filled ages: [25.0, 30.0, 31.0, 28.0, 92.0]
Final: 4 rows
```

6.4 Common mistakes and best practices

Mistake	What happens	Best practice
Creating empty dict with `{}`	Gets a dict, not a set	Use `set()`
Removing duplicates from a list	Loses original order	Use `dict.fromkeys()` or `sorted(set(...))`
Not handling missing data	Wrong calculations	Always check for `None`/`NaN`
In-place modification in a pipe	Confusing state changes	Chain methods or assign results

6.5 Practice exercises

1. Find the common words between two strings (use sets).
2. Remove duplicates from a list while keeping the original order.
3. Clean a dataset: fill missing numeric values, drop rows with missing names.

4. Check if all vowels are present in a string.
5. Count unique elements in a list.

6.6 Solutions

1. `set(words1) & set(words2)`
2. `list(dict.fromkeys(my_list))`
3. `df["col"].fillna(df["col"].mean(), inplace=True); df.dropna(subset=["name"], inplace=True)`
4. `set("aeiou").issubset(set(text.lower()))`
5. `len(set(my_list))`

6.7 Mini test

Questions

1. What is the main property of a set?
2. How does `{}` differ from `set()`?
3. What does `set1 - set2` return?
4. Why are sets faster than lists for membership testing?
5. What is the time complexity of checking `item in my_set`?

Answer key and explanations

1. Sets contain only unique elements (no duplicates).
2. `{}` creates an empty dictionary; `set()` creates an empty set.
3. Items in `set1` that are not in `set2` (set difference).
4. Sets use hash tables; membership testing is O(1) average.
5. O(1) — constant time on average.

Chapter 7 - String manipulation

7.1 Summary

- Strings are immutable sequences of characters.
- Common methods: `upper`, `lower`, `strip`, `split`, `join`, `replace`, `startswith`, `endswith`.
- String formatting: f-strings (preferred), `.format()`, `%`.
- Regular expressions (regex) match patterns in text.
- String indexing and slicing work the same as lists.
- `in` operator checks substring membership.
- `repr()` and `str()` convert objects to strings.

7.2 Concepts

String methods

Python

```python
text = "  Hello, World!  "
print(text.strip())          # "Hello, World!"
print(text.upper())          # "  HELLO, WORLD!  "
print(text.lower())          # "  hello, world!  "
print(text.replace("World", "AI"))  # "  Hello, AI!  "
```

F-strings

Python

```python
name = "Ada"
age = 36
score = 95.5

message = f"{name} is {age} years old and scored {score:.1f}"
print(message)
```

7.3 Examples

Example 1 - Parsing structured strings

Python

```python
# ch07_parsing.py
log_line = "2024-01-15 14:30:22 ERROR Database connection failed"

parts = log_line.split(" ", 3)
date = parts[0]
time = parts[1]
level = parts[2]
message = parts[3]

print(f"Date: {date}")
print(f"Time: {time}")
print(f"Level: {level}")
print(f"Message: {message}")
```

output

```
Date: 2024-01-15
Time: 14:30:22
Level: ERROR
Message: Database connection failed
```

Example 2 - F-strings with formatting

Python

```python
# ch07_fstrings.py
products = [
    ("Laptop", 999.99, 0.1),
    ("Mouse", 29.99, 0.2),
    ("Keyboard", 79.99, 0.15)
]

for name, price, discount in products:
    final = price * (1 - discount)
    print(f"{name:10} ${price:7.2f} -> ${final:7.2f}")
```

output

```
Laptop      $ 999.99 -> $ 899.99
Mouse       $  29.99 -> $  23.99
Keyboard    $  79.99 -> $  67.99
```

Example 3 - Regular expressions

Python

```python
# ch07_regex.py
import re

# Extract email addresses
text = "Contact us at support@example.com or sales@example.com"
emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', text)
print("Found emails:", emails)

# Replace phone numbers with a placeholder
phone_text = "Call 123-456-7890 or 987-654-3210"
masked = re.sub(r'\d{3}-\d{3}-\d{4}', 'XXX-XXX-XXXX', phone_text)
print("Masked:", masked)

# Split by multiple delimiters
items = re.split(r'[,\s;|]+', "apple,banana;cherry date")
print("Items:", items)
```

output

```
Found emails: ['support@example.com', 'sales@example.com']
Masked: Call XXX-XXX-XXXX or XXX-XXX-XXXX
Items: ['apple', 'banana', 'cherry', 'date']
```

7.4 Common mistakes and best practices

Mistake	What happens	Best practice
String concatenation in loops	Slow for large strings	Use `.join()` or list accumulation
Not escaping special regex chars	Wrong matches	Use `re.escape()` or raw strings
Mutable default arguments	Bugs with lists/dicts	Use `None` as default, create inside
Confusing `==` with `is`	Unexpected results	Use `==` for value comparison

7.5 Practice exercises

1. Write a function that returns the initials from a full name.
2. Parse a CSV line: `"Alice,30,Engineer"` into a list.
3. Strip whitespace and convert to title case: `" hello world "`.
4. Count how many times a substring appears in a string.
5. Use regex to validate a simple IP address.

7.6 Solutions

1. `return ''.join(part[0].upper() for part in name.split())`
2. `line.strip().split(",")`
3. `text.strip().title()`
4. `text.count(substring)`
5. `re.match(r'^(\d{1,3}\.){3}\d{1,3}$', ip)`

7.7 Mini test

Questions

1. What does `"abc"[1]` return?
2. What does `"hello world".split()` return?
3. What is an f-string?
4. What does `r"\d+"` mean in regex?
5. Is `"abc" == "abc"` True or False? What about `"abc" is "abc"`?

Answer key and explanations

1. `"b"` — indexing returns the character at that position.
2. `['hello', 'world']` — splits on whitespace by default.
3. A formatted string literal prefixed with `f`, allowing `{variable}` interpolation.
4. A raw string matching one or more digits (`\d` = digit, `+` = one or more).
5. Both are True for short strings (Python interns them), but `is` should not be used for value comparison; `is` checks identity, `==` checks equality.

Chapter 8 - Advanced Python

8.1 Summary

- Error handling uses `try / except / else / finally`.
- Custom exceptions are created by subclassing `Exception`.

- Modules are files; packages are directories of modules.
- Import styles: `import module`, `from module import function`.
- File I/O uses `open()` with context managers (`with`).
- Standard library highlights: `os`, `sys`, `json`, `pathlib`, `datetime`.
- `__name__ == "__main__"` runs code only when the file is executed directly.

8.2 Concepts

Exception handling

Python

```python
try:
    value = int(input("Enter a number: "))
    result = 10 / value
    print(f"Result: {result}")
except ValueError:
    print("That wasn't a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Calculation succeeded.")
finally:
    print("Done.")
```

8.3 Examples

Example 1 - Context managers for file I/O

Python

```python
# ch08_file_io.py
from pathlib import Path

# Write to a file
output_path = Path("output.txt")
with open(output_path, "w") as f:
    f.write("Hello, World!\n")
    f.write("Line 2 here.\n")

# Read it back
with open(output_path, "r") as f:
    content = f.read()
    print("File contents:")
    print(content)
```

output

```
File contents:
Hello, World!
Line 2 here.
```

Example 2 - Working with modules

Python

```python
# ch08_modules.py
import math
import datetime as dt
from pathlib import Path

# Math functions
print(f"sqrt(16) = {math.sqrt(16)}")
print(f"pi = {math.pi:.4f}")

# Date and time
today = dt.date.today()
print(f"Today is {today:%A, %B %d, %Y}")

# File paths
current_dir = Path(".")
files = list(current_dir.glob("*.py"))
print(f"Python files here: {[f.name for f in files]}")
```

output

```
sqrt(16) = 4.0
pi = 3.1416
Today is Sunday, October 11, 2026
Python files here: ['ch08_modules.py']
```

Example 3 - Custom exception

Python

```python
# ch08_custom_exception.py
class ValidationError(Exception):
    """Raised when input data fails validation."""
    pass

def validate_age(age):
    if not isinstance(age, int):
        raise TypeError("Age must be an integer")
    if age < 0 or age > 150:
        raise ValidationError(f"Age {age} is out of valid range")
    return True

for test_age in [25, -5, 200, "thirty"]:
    try:
        validate_age(test_age)
        print(f"Age {test_age} is valid")
    except (TypeError, ValidationError) as e:
        print(f"Age {test_age} is invalid: {e}")
```

output

```
Age 25 is valid
Age -5 is invalid: Age -5 is out of valid range
Age 200 is invalid: Age 200 is out of valid range
Age thirty is invalid: Age must be an integer
```

8.4 Common mistakes and best practices

Mistake	What happens	Best practice
Bare `except`	Catches everything, hides bugs	Catch specific exceptions
Not closing files	File handles leak	Use `with` statement
Circular imports	`ImportError`	Restructure modules
Not handling `None`	`AttributeError`	Check for `None` before use

8.5 Practice exercises

1. Write a function that reads a JSON file and returns the parsed data.
2. Create a custom exception for invalid input.
3. Write a function that lists all files in a directory recursively.
4. Handle division by zero gracefully in a program.
5. Use `pathlib` to check if a file exists before writing to it.

8.6 Solutions

1. `with open(path) as f: return json.load(f)`
2. `class MyError(Exception): pass; raise MyError("message")`
3. `Path(dir).rglob("*")` or `os.walk(dir)`
4. `try: result = a / b except ZeroDivisionError: result = None`
5. `if not Path(path).exists(): Path(path).write_text(data)`

8.7 Mini test

Questions

1. What does `try/except` do?
2. What is the purpose of `finally`?
3. How do you import only a specific function from a module?
4. What does `with open(...) as f:` do?
5. What is the `__name__` variable?

Answer key and explanations

1. It catches and handles exceptions (errors) that occur during execution.
2. `finally` runs no matter what, even if an exception was raised.
3. `from module import function`.
4. It opens a file and ensures it is closed when the block exits.
5. It is `"__main__"` when the file is run directly, otherwise the module name.

Chapter 9 - Data science with pandas

9.1 Summary

- Pandas provides DataFrame and Series for tabular data.
- Read CSV with `pd.read_csv()`.
- Inspect with `df.head()`, `df.info()`, `df.describe()`.
- Select columns: `df["col"]` or `df.col`.
- Filter rows with boolean indexing: `df[df["col"] > 50]`.
- Group by: `df.groupby("col").mean()`.
- Handle missing data with `df.dropna()` and `df.fillna()`.

9.2 Concepts

What is a DataFrame?

A DataFrame is a two-dimensional table (like a spreadsheet or SQL table) with rows and columns. Each column can have a different data type.

Python

```python
import pandas as pd

df = pd.DataFrame({
    "name": ["Alice", "Bob", "Carol"],
    "score": [85, 92, 78],
    "passed": [True, True, False]
})
print(df)
```

9.3 Examples

Example 1 - Loading and inspecting data

Python

```python
# ch09_pandas_basics.py
import pandas as pd
from io import StringIO

csv_data = """name,age,score,city
Alice,25,85,New York
Bob,30,92,London
Carol,35,78,Paris
Diana,28,None,Berlin
Eve,40,95,London
"""

df = pd.read_csv(StringIO(csv_data))
print("Shape:", df.shape)
print("\nFirst 3 rows:")
print(df.head(3))
print("\nInfo:")
df.info()
```

output

```
Shape: (5, 4)
First 3 rows:
    name  age  score    city
0  Alice   25   85.0  New York
1    Bob   30   92.0    London
2  Carol   35   78.0     Paris

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   name    5 non-null    object
 1   age     5 non-null    int64
 2   score   5 non-null    float64
 3   city    5 non-null    object
dtypes: 2 non-null objects, 1 non-null int, 1 non-null float
```

Example 2 - Filtering and grouping

Python

```python
# ch09_filter_group.py
import pandas as pd
from io import StringIO

csv_data = """name,city,score
Alice,New York,85
Bob,London,92
Carol,Paris,78
Diana,Berlin,88
Eve,London,95
Frank,Paris,82
Grace,London,90
"""

df = pd.read_csv(StringIO(csv_data))

# Filter rows
london_high = df[(df["city"] == "London") & (df["score"] >= 90)]
print("London high scorers:")
print(london_high)

# Group by city and compute statistics
by_city = df.groupby("city")["score"].agg(["count", "mean", "max"])
print("\nScore by city:")
print(by_city)
```

output

```
London high scorers:
  name    city  score
1    Bob  London     92
6  Grace  London     90

Score by city:
           count  mean    max
city
Berlin         1  88.0   88.0
London         3  92.3   95.0
New York       1  85.0   85.0
Paris          2  80.0   82.0
```

Example 3 - Handling missing data

Python

```python
# ch09_missing.py
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "A": [1, 2, np.nan, 4],
    "B": ["x", None, "z", "w"],
    "C": [10, 20, 30, np.nan]
})

print("Original:")
print(df)

# Drop rows with any missing values
dropped = df.dropna()
print("\nDropped rows with NaN:")
print(dropped)

# Fill missing values
filled = df.copy()
filled["A"] = filled["A"].fillna(filled["A"].mean())
filled["C"] = filled["C"].fillna(filled["C"].median())
filled["B"] = filled["B"].fillna("unknown")
print("\nFilled:")
print(filled)
```

output

```
Original:
     A    B     C
0  1.0    x  10.0
1  2.0  NaN  20.0
2  NaN    z  30.0
3  4.0    w   NaN

Dropped rows with NaN:
     A    B     C
0  1.0    x  10.0

Filled:
     A       B     C
0  1.0    x  10.0
1  2.0  unknown  20.0
2  2.33      z  30.0
3  4.0    w  23.33
```

9.4 Common mistakes and best practices

Mistake	What happens	Best practice
Chained indexing	Unexpected results, SettingWithCopyWarning	Use `.loc` for setting values
Not using `.copy()`	Modifying views of the original DataFrame	`df.copy()` before filtering
Comparing with NaN	`NaN != NaN` is always True	Use `df.isna()` or `df.notna()`
Inefficient loops	Slow	Use vectorised operations

9.5 Practice exercises

1. Load a CSV file and show the first 5 rows.
2. Filter rows where a numeric column is above its mean.
3. Group by a categorical column and count rows in each group.
4. Fill missing values with the median for numeric columns.
5. Sort by a column in descending order and show the top 3.

9.6 Solutions

1. `df = pd.read_csv("file.csv"); print(df.head())`
2. `df[df["col"] > df["col"].mean()]`
3. `df.groupby("cat").size()`
4. `df["num"].fillna(df["num"].median(), inplace=True)`
5. `df.sort_values("col", ascending=False).head(3)`

9.7 Mini test

Questions

1. What is a pandas DataFrame?
2. How do you select a single column?
3. What does `df.head()` do?
4. How do you group data in pandas?
5. What is `df.dropna()` used for?

Answer key and explanations

1. A two-dimensional labelled data structure with rows and columns.
2. `df["column_name"]` or `df.column_name`.
3. It shows the first 5 rows of the DataFrame.
4. `df.groupby("column")` — then apply an aggregation function.
5. It drops rows with missing (NaN) values.

Chapter 10 - Numpy arrays

10.1 Summary

- NumPy provides `ndarray`, a fast, vectorised array structure.
- Arrays have a shape and a data type; all elements share one type.
- Vectorised operations apply to entire arrays without loops.

- Common functions: `np.array`, `np.arange`, `np.zeros`, `np.linspace`.
- Broadcasting lets arrays of different shapes operate together.
- Indexing/slicing works like lists but supports N dimensions.
- Universal functions (`ufuncs`) like `np.sqrt`, `np.exp` operate element-wise.

10.2 Concepts

Creating arrays

Python

```python
import numpy as np

# From a list
arr = np.array([1, 2, 3, 4])

# Ranges
a = np.arange(0, 10, 2)   # [0, 2, 4, 6, 8]
b = np.linspace(0, 1, 5)  # [0, 0.25, 0.5, 0.75, 1.0]

# Special arrays
zeros = np.zeros((3, 3))   # 3x3 array of zeros
ones = np.ones(5)          # [1, 1, 1, 1, 1]
```

10.3 Examples

Example 1 - Vectorised operations

Python

```python
# ch10_numpy_ops.py
import numpy as np

# Create two arrays
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

# Element-wise operations (no loops needed)
print("Addition:", a + b)
print("Subtraction:", b - a)
print("Multiplication:", a * b)
print("Division:", b / a)

# Mathematical functions
print("Square root:", np.sqrt(a))
print("Exponential:", np.exp(a))
print("Logarithm:", np.log(np.array([1, 10, 100])))
```

output

```
Addition: [11 22 33 44 55]
Subtraction: [ 9 18 27 36 45]
Multiplication: [ 10  40  90 160 250]
Division: [10. 10. 10. 10. 10.]
Square root: [1.         1.41421356 1.73205081 2.         2.23606805]
Exponential: [  2.71828183   7.3890551   20.08553692  54.59815003 148.4131591 ]
Logarithm: [0.         2.30258509 4.60517018]
```

Example 2 - 2D arrays and slicing

Python

```python
# ch10_2d.py
import numpy as np

# Create a 2D array (matrix)
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print("Shape:", matrix.shape)
print("Element [1,2]:", matrix[1, 2])  # 6
print("Row 1:", matrix[1, :])          # [4 5 6]
print("Column 2:", matrix[:, 2])        # [3 6 9]
print("Sub-matrix:\n", matrix[:2, 1:])

# Reshaping
flattened = matrix.flatten()
print("Flattened:", flattened)
reshaped = flattened.reshape(1, 9)
print("Reshaped to 1x9:\n", reshaped)
```

output

```
Shape: (3, 3)
Element [1,2]: 6
Row 1: [4 5 6]
Column 2: [3 6 9]
Sub-matrix:
 [[2 3]
 [5 6]]
Flattened: [1 2 3 4 5 6 7 8 9]
Reshaped to 1x9:
 [[1 2 3 4 5 6 7 8 9]]
```

Example 3 - Broadcasting

Python

```python
# ch10_broadcasting.py
import numpy as np

# 3x3 matrix
M = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

# Add a scalar (broadcasts to all elements)
print("M + 10:\n", M + 10)

# Add a vector (broadcasts along rows)
row_vec = np.array([10, 20, 30])
print("M + row_vec:\n", M + row_vec)

# Add a column vector (broadcasts along columns)
col_vec = np.array([[100], [200], [300]])
print("M + col_vec:\n", M + col_vec)

# Statistical functions
print("\nMean of each row:", M.mean(axis=1))
print("Sum of each column:", M.sum(axis=0))
print("Overall max:", M.max())
```

output

```
M + 10:
 [[11 12 13]
 [14 15 16]
 [17 18 19]]
M + row_vec:
 [[11 22 33]
 [14 25 36]
 [17 28 39]]
M + col_vec:
 [[101 102 103]
 [204 205 206]
 [307 308 309]]

Mean of each row: [2. 5. 8.]
Sum of each column: [12 15 18]
Overall max: 9
```

10.4 Common mistakes and best practices

Mistake	What happens	Best practice
Using lists for numeric data	Slow, no vectorisation	Use numpy arrays
Forgetting to set a dtype	Unexpected type upcasting	`np.array(data, dtype=np.float32)`
Indexing without understanding	Hard-to-find bugs	Remember numpy uses 0-based indexing

Mistake	What happens	Best practice
Not using axis parameter	Wrong aggregation direction	Explicitly set `axis=0` (rows) or `axis=1` (columns)

10.5 Practice exercises

1. Create an array of 100 random numbers and compute mean and std.
2. Create a 5x5 identity matrix.
3. Given a 1D array, find the indices of elements above 5.
4. Normalise a numeric array to range [0, 1].
5. Compute the dot product of two vectors.

10.6 Solutions

1. `arr = np.random.rand(100); print(arr.mean(), arr.std())`
2. `np.eye(5)`
3. `np.where(arr > 5)[0]`
4. `(arr - arr.min()) / (arr.max() - arr.min())`
5. `np.dot(v1, v2)` or `v1 @ v2`

10.7 Mini test

Questions

1. What is the main advantage of numpy arrays over Python lists?
2. What does `np.zeros((3, 3))` create?
3. What is broadcasting?
4. What does `axis=0` mean in aggregation functions?
5. What does `arr.shape` return for a 2D array?

Answer key and explanations

1. Vectorised operations are much faster (implemented in C).
2. A 3x3 array filled with zeros.
3. Automatic expansion of smaller arrays to match larger ones in operations.
4. `axis=0` aggregates along rows (produces one value per column).
5. A tuple showing the dimensions (rows, columns).

Chapter 11 - Data visualisation

11.1 Summary

- Matplotlib is the base plotting library in Python.
- Seaborn makes attractive statistical plots with less code.
- Common plot types: line, bar, histogram, scatter, box.
- Key parameters: `figsize`, `title`, `xlabel`, `ylabel`, `color`, `alpha`.
- The matplotlib object model: `fig, ax = plt.subplots()`.
- Saving figures: `fig.savefig("plot.png", dpi=150)`.
- Choosing the right plot for the data type.

11.2 Concepts

The matplotlib object model

Python

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot([1, 2, 3], [4, 5, 3])
ax.set_title("My Plot")
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
fig.savefig("plot.png", dpi=150)
```

11.3 Examples

Example 1 - Line plot

Python

```python
# ch11_line_plot.py
import matplotlib
matplotlib.use("Agg")  # non-interactive backend
import matplotlib.pyplot as plt
import numpy as np

# Simulate data
x = np.linspace(0, 10, 100)
y_sin = np.sin(x)
y_cos = np.cos(x)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y_sin, label="sin(x)", color="blue", linewidth=2)
ax.plot(x, y_cos, label="cos(x)", color="red", linestyle="--")
ax.set_title("Sine and Cosine Functions")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
ax.grid(True, alpha=0.3)
fig.savefig("triganometry.png", dpi=150, bbox_inches="tight")
print("Saved triganometry.png")
```

Example 2 - Histogram

Python

```python
# ch11_histogram.py
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

# Simulate exam scores with a normal distribution
np.random.seed(42)
scores = np.random.normal(75, 15, 500)
scores = np.clip(scores, 0, 100)  # clip to valid range

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(scores, bins=20, color="steelblue", edgecolor="white", alpha=0.8)
ax.axvline(75, color="red", linestyle="--", label="Mean")
ax.set_title("Distribution of Exam Scores")
ax.set_xlabel("Score")
ax.set_ylabel("Number of students")
ax.legend()
fig.savefig("histogram.png", dpi=150)
print(f"Mean: {scores.mean():.1f}, Std: {scores.std():.1f}")
```

output

```
Mean: 75.0, Std: 13.9
```

Example 3 - Scatter plot with seaborn

Python

```python
# ch11_scatter.py
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Generate correlated data
np.random.seed(42)
n = 200
x = np.random.uniform(0, 100, n)
y = 2.5 * x + np.random.normal(0, 15, n)

fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(x=x, y=y, ax=ax, alpha=0.6, s=50)
ax.set_title("Study Time vs Score")
ax.set_xlabel("Hours studied")
ax.set_ylabel("Exam score")
fig.savefig("scatter.png", dpi=150)
print("Saved scatter.png")
```

Example 4 - Bar chart

Python

```python
# ch11_bar_chart.py
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

categories = ["Python", "JavaScript", "Java", "C++", "Go"]
votes = [85, 72, 58, 45, 32]

fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.bar(categories, votes, color="coral", edgecolor="black")
ax.set_title("Developer Survey: Language Preference")
ax.set_ylabel("Votes (%)")
ax.set_ylim(0, 100)

# Add value labels on bars
for bar, vote in zip(bars, votes):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            str(vote), ha="center", va="bottom")
fig.savefig("bar_chart.png", dpi=150)
print("Saved bar_chart.png")
```

11.4 Common mistakes and best practices

Mistake	What happens	Best practice
Plotting without labels	Chart is meaningless	Always set title, xlabel, ylabel
Too many colors	Distracting	Use 3-4 colors max, consistent palette
Default figure size	Plot too small/large	Set `figsize` explicitly
Overlapping labels	Hard to read	Rotate labels, adjust layout
Not saving properly	Empty file	Call `fig.savefig()` before `plt.show()`

11.5 Practice exercises

1. Create a line plot showing temperature over a week.
2. Plot a histogram of 1000 random numbers from a uniform distribution.
3. Create a bar chart comparing sales for 5 products.
4. Make a scatter plot with a trend line.
5. Save your plot as a PNG with 300 DPI.

11.6 Solutions

1. Use `np.array` for days 1-7 and temperatures; `ax.plot(days, temps)`
2. `data = np.random.uniform(0, 1, 1000); ax.hist(data, bins=30)`
3. `ax.bar(products, sales, color="steelblue")`
4. Use `sns.regplot(x=x, y=y, scatter=True)`
5. `fig.savefig("plot.png", dpi=300, bbox_inches="tight")`

11.7 Mini test

Questions

1. Which library is built on top of matplotlib for statistical plots?
2. What plot type shows the distribution of a single variable?
3. What does `plt.subplots()` return?
4. How do you save a matplotlib figure?
5. What is the default color cycle in matplotlib?

Answer key and explanations

1. Seaborn (or `seaborn`).
2. A histogram.
3. A tuple `(fig, ax)` — the Figure and Axes objects.
4. `fig.savefig("filename.png")`.
5. Blue, orange, green, red, purple, brown, pink, gray, olive, cyan.

Chapter 12 - Feature engineering

12.1 Summary

- Feature engineering transforms raw data into model-ready features.
- Numerical features often need scaling (min-max or standard).
- Categorical features need encoding (one-hot, label, target).
- Missing values can be imputed or dropped.
- Feature interactions create new features from existing ones.
- Date/time features can be decomposed (year, month, day, hour).
- Text features can be bag-of-words or TF-IDF vectors.

12.2 Concepts

Scaling numerical features

Machine learning models often work better when numerical features are on a similar scale. Two common techniques:

- Min-Max scaling: maps values to [0, 1]. Formula: `(x - min) / (max - min)`
- Standard scaling: maps values to mean 0, std 1. Formula: `(x - mean) / std`

12.3 Examples

Example 1 - One-hot encoding

Python

```python
# ch12_one_hot.py
import pandas as pd

df = pd.DataFrame({
    "color": ["red", "blue", "green", "red", "blue"],
    "size": ["S", "M", "L", "M", "L"],
    "price": [10, 20, 30, 15, 25]
})

# One-hot encode categorical columns
encoded = pd.get_dummies(df, columns=["color", "size"], prefix=["color", "size"])
print("One-hot encoded:")
print(encoded)
print("\nColumn names:", encoded.columns.tolist())
```

output

```
One-hot encoded:
    price  color_blue  color_green  color_red  size_L  size_M  size_S
0      10           0            0          1       0       0       1
1      20           1            0          0       0       1       0
2      30           0            1          0       1       0       0
3      15           0            0          1       0       1       0
4      25           1            0          0       1       0       0

Column names: ['price', 'color_blue', 'color_green', 'color_red', 'size_L', 'size_M', 'size_S']
```

Example 2 - Scaling features

Python

```python
# ch12_scaling.py
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import numpy as np

np.random.seed(42)
df = pd.DataFrame({
    "age": np.random.randint(18, 80, 100),
    "income": np.random.normal(50000, 15000, 100),
    "score": np.random.uniform(0, 100, 100)
})

# Min-Max scaling
scaler_minmax = MinMaxScaler()
df_minmax = pd.DataFrame(
    scaler_minmax.fit_transform(df),
    columns=df.columns
)
print("Min-Max scaled (first 3 rows):")
print(df_minmax.head(3))

# Standard scaling
scaler_std = StandardScaler()
df_std = pd.DataFrame(
    scaler_std.fit_transform(df),
    columns=df.columns
)
print("\nStandard scaled (first 3 rows):")
print(df_std.head(3).round(3))
```

output

```
Min-Max scaled (first 3 rows):
   age    income     score
0  0.171  0.526525  0.640924
1  0.357  0.476636  0.201694
2  0.714  0.612831  0.685114

Standard scaled (first 3 rows):
         age    income     score
0  -0.048  -0.023  -0.068
1   0.327  -0.168   0.223
2   1.521  -0.078   0.073
```

Example 3 - Missing value imputation

Python

```python
# ch12_imputation.py
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

df = pd.DataFrame({
    "age": [25, 30, np.nan, 40, 35],
    "income": [50000, np.nan, 70000, np.nan, 60000],
    "city": ["NYC", "London", "Paris", None, "London"]
})

print("Original:")
print(df)

# Impute numeric with median
num_imputer = SimpleImputer(strategy="median")
df[["age", "income"]] = num_imputer.fit_transform(df[["age", "income"]])

# Impute categorical with most frequent
cat_imputer = SimpleImputer(strategy="most_frequent")
df[["city"]] = cat_imputer.fit_transform(df[["city"]])

print("\nAfter imputation:")
print(df)
```

output

```
Original:
    age      income    city
0  25.0  50000.0     NYC
1  30.0        NaN  London
2   NaN  70000.0     Paris
3  40.0        NaN     NaN
4  35.0  60000.0  London

After imputation:
     age    income    city
0  25.0  50000.0     NYC
1  30.0  60000.0  London
2  35.0  70000.0     Paris
3  40.0  60000.0  London
4  35.0  60000.0  London
```

12.4 Common mistakes and best practices

Mistake	What happens	Best practice
Scaling before train/test split	Data leakage	Fit scaler on train only, transform test
One-hot encoding before split	Inconsistent categories	Fit encoder on train only
Dropping missing values	Lose data	Impute when possible
Creating too many features	Overfitting	Keep features interpretable, use feature selection

12.5 Practice exercises

1. Create a synthetic dataset and one-hot encode a categorical column.
2. Scale a numeric column using min-max scaling.
3. Fill missing values in a column with its mean.
4. Create an interaction feature (product of two columns).
5. Extract the day of week from a date column.

12.6 Solutions

1. `pd.get_dummies(df, columns=["category"])`
2. `df["scaled"] = (df["col"] - df["col"].min()) / (df["col"].max() - df["col"].min())`
3. `df["col"].fillna(df["col"].mean(), inplace=True)`
4. `df["interaction"] = df["col1"] * df["col2"]`
5. `df["day_of_week"] = pd.to_datetime(df["date"]).dt.day_name()`

12.7 Mini test

Questions

1. What is feature engineering?
2. When should you scale features?
3. What is one-hot encoding used for?
4. What strategy should you use for imputation: train only or train+test?
5. What is a feature interaction?

Answer key and explanations

1. Transforming raw data into features suitable for machine learning models.
2. When the model is sensitive to scale (e.g., k-NN, linear models, neural nets).
3. Converting categorical variables into binary (0/1) columns.
4. Train only — fitting on test data causes data leakage.
5. A new feature created by combining two or more existing features.

Chapter 13 - End-to-end project

13.1 Summary

- An end-to-end ML project follows a structured workflow.
- Steps: define the problem, collect/load data, explore, clean, engineer features, choose a model, train, evaluate, iterate, deploy.

- Always set a random seed for reproducibility.
- Split data into train, validation and test sets.
- Use cross-validation to get a robust performance estimate.
- Compare multiple models, not just the first one you try.
- Document everything: assumptions, decisions, results.
- The final test set is used only once, at the very end.

13.2 Concepts

The ML workflow

```
Problem -> Data -> EDA -> Cleaning -> Features -> Model -> Evaluate -> Deploy
```

Each step builds on the previous one. Skipping steps leads to bugs and data leakage.

13.3 Examples

Example 1 - Loading and exploring a dataset

Python

```python
# ch13_titanic.py
import pandas as pd
from sklearn.model_selection import train_test_split

# Load the dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print("Shape:", df.shape)
print("\nFirst few rows:")
print(df.head())

print("\nMissing values:")
print(df.isnull().sum())

print("\nSurvival rate by class:")
print(df.groupby("Pclass")["Survived"].mean())
```

output

```
Shape: (891, 12)
Missing values:
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin            687
Embarked         2

Survival rate by class:
Pclass
1    0.627817
2    0.472828
3    0.242363
```

Example 2 - Feature engineering and model training

Python

```python
# ch13_model.py
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix

# Load and prepare
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Feature engineering
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

# Fill missing age with median
df["Age"] = df["Age"].fillna(df["Age"].median())

# Encode categorical variables
le = LabelEncoder()
df["SexCode"] = le.fit_transform(df["Sex"])
df["Embarked"] = df["Embarked"].fillna("S")
df["EmbarkedCode"] = le.fit_transform(df["Embarked"])

# Select features
features = ["Pclass", "SexCode", "Age", "Fare", "FamilySize", "IsAlone", "EmbarkedCode"]
X = df[features]
y = df["Survived"]

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print(f"Test accuracy: {accuracy_score(y_test, y_pred):.3f}")

# Cross-validation
cv_scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
print(f"CV accuracy: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.3f})")

# Feature importance
for feat, imp in zip(features, model.feature_importances_):
    print(f"  {feat}: {imp:.3f}")
```

output

```
Test accuracy: 0.810
CV accuracy: 0.815 (+/- 0.065)
  Pclass: 0.243
  SexCode: 0.205
  Age: 0.164
  Fare: 0.121
  FamilySize: 0.097
  IsAlone: 0.073
  EmbarkedCode: 0.031
```

Example 3 - Submitting predictions

Python

```python
# ch13_submit.py
import pandas as pd

# Load test data
test_df = pd.read_csv("test.csv")

# Same feature engineering as training
test_df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
test_df["IsAlone"] = (test_df["FamilySize"] == 1).astype(int)
test_df["Age"] = test_df["Age"].fillna(df["Age"].median())
test_df["SexCode"] = le.transform(test_df["Sex"])
test_df["Embarked"] = test_df["Embarked"].fillna("S")
test_df["EmbarkedCode"] = le.transform(test_df["Embarked"])

X_test = test_df[features]
predictions = model.predict(X_test)

# Create submission file
submission = pd.DataFrame({
    "PassengerId": test_df["PassengerId"],
    "Survived": predictions
})
submission.to_csv("submission.csv", index=False)
print("Saved submission.csv")
print(submission.head())
```

output

```
Saved submission.csv
   PassengerId  Survived
0          892         0
1          893         0
2          894         0
3          895         1
4          896         0
```

13.4 Common mistakes and best practices

Mistake	What happens	Best practice
Using the test set for model selection	Overoptimistic results	Use a separate validation set or CV
Not setting random seeds	Non-reproducible results	`random_state=42` everywhere
Data leakage	Unrealistic performance	Split before feature engineering
Ignoring class imbalance	Biased predictions	Use stratified splits, class weights
Not checking feature importance	Misses insights	Always inspect what the model learned

13.5 Practice exercises

1. Choose a dataset from sklearn and load it.

2. Perform basic EDA: shapes, missing values, distributions.
3. Engineer at least two new features.
4. Train a model and evaluate with cross-validation.
5. Create a submission file with predictions.

13.6 Solutions

1. `from sklearn.datasets import load_iris; data = load_iris()`
2. `df.info(); df.describe(); df.isnull().sum()`
3. Interaction features, polynomial features, binning
4. `cross_val_score(model, X, y, cv=5)`
5. `submission.to_csv("submission.csv", index=False)`

13.7 Mini test

Questions

1. What is the purpose of the test set?
2. Why should you split before feature engineering?
3. What does cross-validation give you?
4. What is data leakage?
5. How do you ensure reproducibility?

Answer key and explanations

1. To get an unbiased estimate of model performance on unseen data.
2. To prevent information from the test set leaking into the training process.
3. A more robust estimate of model performance by averaging over multiple splits.
4. When information from outside the training set influences the model, leading to unrealistically optimistic results.
5. Set `random_state=42` (or any fixed seed) in all random operations.

Chapter 14 - Where to go next

14.1 Summary

- This chapter lists the next steps after completing the Python for AI path.
- Deepen your Python: decorators, generators, context managers.
- Learn version control (Git) for reproducibility.
- Explore more Kaggle micro-courses: pandas, data visualisation, SQL, ML, DL.

- Read books: "Python for Data Analysis" (McKinney), "Hands-On ML" (Geron).
- Practice on Kaggle competitions to learn from others' notebooks.

14.2 Concepts

Continuing the journey

Resource	Focus
Kaggle Learn	micro-courses in pandas, SQL, ML, DL
Python docs	official documentation, advanced features
Real projects	clone existing analyses, find datasets on Kaggle
Communities	Kaggle forums, Reddit r/learnpython, r/datascience

14.3 Examples

Example 1 - Decorators (advanced)

Python

```python
# ch14_decorator.py
import time

def timer(func):
    """A decorator that measures execution time."""
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f"{func.__name__} ran in {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    total = sum(range(1000000))
    return total

slow_function()
```

output

```
slow_function ran in 0.0123s
```

14.4 Common mistakes

Mistake	What happens	Best practice
Trying to learn everything at once	Overwhelm	Focus on one topic at a time

Mistake	What happens	Best practice
Not writing your own code	Not learning	Always code along, don't just read
Not practicing	Forgetting	Solve exercises daily
Copying code without understanding	No progress	Write explanations in your own words

14.5 Practice exercises

1. Choose one Kaggle micro-course to complete next.
2. Set up Git and version-control one of your projects.
3. Join a data science community and introduce yourself.
4. Find a dataset you're interested in and plan an analysis.

14.6 Mini test

Questions

1. What is the most important skill for a data scientist?
2. Should you copy-paste code from tutorials?
3. How often should you practice coding?
4. What is the next Kaggle course to take?
5. Why is version control important?

Answer key and explanations

1. Problem-solving and the ability to learn independently.
2. No — type it yourself to understand each line.
3. Daily, even if only for 30 minutes.
4. Depends on your interests, but "Pandas" or "Data Visualization" are good next steps.
5. It tracks changes, enables collaboration, and makes your work reproducible.

Cheat sheet - Python for data science

Concept	Syntax
Variable	`x = 10`
Print	`print(x)`
Comment	`# comment`
String	`"hello"` or `'hello'`

Concept	Syntax
F-string	`f"Value: {x}"`
If/elif/else	`if x > 0: ...`
For loop	`for i in range(10): ...`
While loop	`while x < 10: ...`
Function	`def my_func(a, b): return a + b`
List	`[1, 2, 3]`
Dict	`{"key": "value"}`
Set	`{1, 2, 3}`
Import	`import numpy as np`
Try/except	`try: ... except: ...`

15.1 Libraries quick reference

Library	Alias	Use
NumPy	np	arrays, math
pandas	pd	DataFrames
matplotlib	plt	plots
seaborn	sns	statistical plots
scikit-learn	sklearn	ML models

Final test

1. What is the output of `print(10 // 3)`?
2. What does `len([1, 2, 3, 4, 5])` return?
3. What keyword defines a function?
4. What does `dict.get("key", "default")` return if "key" is not present?
5. What is the purpose of the `for` loop?
6. How do you create a numpy array from a list?
7. What does `df.head(5)` do?
8. What is the difference between `list.append()` and `list.extend()`?
9. What does `import pandas as pd` do?
10. What is the result of `"hello".upper()`?

11. What does `if __name__ == "__main__":` do?
12. How do you read a CSV file with pandas?
13. What does `np.array([1, 2, 3]).shape` return?
14. What is a list comprehension?
15. How do you handle an exception in Python?

16.1 Final test answers

1. `3` — Integer division discards the remainder.
2. `5` — Length is the number of items.
3. `def`.
4. `"default"` — The default argument is returned when the key is not found.
5. It repeats a block of code for each item in a sequence.
6. `np.array([1, 2, 3])`.
7. It shows the first 5 rows of a DataFrame.
8. `append` adds a single item; `extend` adds all items from another list.
9. It imports the pandas library and gives it the alias `pd`.
10. `"HELLO"`.
11. It ensures the code only runs when the file is executed directly as a script.
12. `pd.read_csv("filename.csv")`.
13. `(3,)` — A 1D array with 3 elements.
14. A concise way to create a list: `[expression for item in iterable]`.
15. Using `try/except` blocks.

Sources

- https://www.kaggle.com/learn/python
- https://docs.python.org/3/tutorial/
- https://numpy.org/doc/stable/user/quickstart.html
- https://pandas.pydata.org/docs/getting_started/index.html
- https://matplotlib.org/stable/tutorials/index.html